

Lab Assignment – 1

CPU Scheduling Algorithms: FCFS & SJF

Course: Operating System Lab (ENCA252)

Program: BCA (AI & DS) (Research)

Task 1: Process Creation & Input Handling

A Process class is defined with attributes PID, AT, BT. Five processes are accepted as input and displayed in a formatted table.

```
class Process:
    """Represents a single process with scheduling attributes."""
    def __init__(self, pid, arrival_time, burst_time):
        self.pid = pid # Process ID
        self.at = arrival_time # Arrival Time
        self.bt = burst_time # Burst Time
        self.ct = 0 # Completion Time
        self.tat = 0 # Turnaround Time
        self.wt = 0 # Waiting Time
        self.start_time = 0 # For Gantt chart
    def get_process_input():
        processes = []
        n = int(input("Enter number of processes (min 4): "))
        for i in range(n):
            pid = int(input(f" PID : "))
            at = int(input(f" Arrival Time : "))
            bt = int(input(f" Burst Time : "))
            processes.append(Process(pid, at, bt))
        return processes
    def display_input_table(processes):
        print(f" {'PID':<8} {'Arrival Time':<16} {'Burst Time':<10}")
        print(" " + "-" * 40)
        for p in processes:
            print(f" {p.pid:<8} {p.at:<16} {p.bt:<10}")
```

```
#####
CPU Scheduling Algorithms: FCFS & SJF
OS Lab Assignment-1 | ENCA252
#####

=====
TASK 1: Process Creation & Input Handling
=====

Enter number of processes (min 4): 5

--- Process 1 ---
PID      : 1
Arrival Time : 0
Burst Time  : 8

--- Process 2 ---
PID      : 2
Arrival Time : 1
Burst Time  : 4

--- Process 3 ---
PID      : 3
Arrival Time : 2
Burst Time  : 9

--- Process 4 ---
PID      : 4
Arrival Time : 3
Burst Time  : 5

--- Process 5 ---
PID      : 5
Arrival Time : 4
Burst Time  : 2

Process Input Table:
-----
PID      Arrival Time  Burst Time
-----
1         0             8
2         1             4
3         2             9
4         3             5
5         4             2
-----
```

Task 2: FCFS Scheduling

Processes are sorted by Arrival Time. CT, TAT and WT are computed sequentially. CPU idle time is handled when no process has arrived yet.

```
def fcfs_scheduling(processes):
    """First Come First Serve - Non-Preemptive"""
    import copy
    procs = copy.deepcopy(processes)
    procs.sort(key=lambda p: p.at) # Sort by Arrival Time
    current_time = 0
    gantt = []
    for p in procs:
        # Handle CPU idle time
        if current_time < p.at:
            gantt.append(("IDLE", current_time, p.at))
            current_time = p.at
```

```

p.start_time = current_time
p.ct = current_time + p.bt # Completion Time
p.tat = p.ct - p.at # Turnaround Time = CT - AT
p.wt = p.tat - p.bt # Waiting Time = TAT - BT
gantt.append(("P{p.pid}", current_time, p.ct))
current_time = p.ct
return procs, gantt

```

TASK 2: FCFS Scheduling Results					
PID	AT	BT	CT	TAT	WT
1	0	8	8	8	0
2	1	4	12	11	7
3	2	9	21	19	10
4	3	5	26	23	18
5	4	2	28	24	22

Task 3: SJF Scheduling (Non-Preemptive)

At each step, the arrived process with the shortest Burst Time is selected. Ties are broken by PID. CT, TAT and WT are computed for each process.

```

def sjf_scheduling(processes):
    """Shortest Job First - Non-Preemptive"""
    import copy
    procs = copy.deepcopy(processes)
    n = len(procs)
    done = [False] * n
    current_time = 0
    gantt = []
    completed = []
    for _ in range(n):
        # Build ready queue: arrived and not yet executed
        ready = [(i, procs[i]) for i in range(n)
                  if not done[i] and procs[i].at <= current_time]
        if not ready:
            # CPU idle – jump to next arrival
            next_at = min(procs[i].at for i in range(n) if not done[i])
            gantt.append(("IDLE", current_time, next_at))
            current_time = next_at
        ready = [(i, procs[i]) for i in range(n)
                  if not done[i] and procs[i].at <= current_time]
        # Pick shortest burst time (ties broken by PID)
        idx, p = min(ready, key=lambda x: (x[1].bt, x[1].pid))
        p.start_time = current_time
        p.ct = current_time + p.bt
        p.tat = p.ct - p.at

```

```

p.wt = p.tat - p.bt
gantt.append((f"P{p.pid}", current_time, p.ct))
current_time = p.ct
done[idx] = True
completed.append(p)
return completed, gantt

```

```

=====
TASK 3: SJF Scheduling Results
=====

```

PID	AT	BT	CT	TAT	WT
1	0	8	8	8	0
5	4	2	10	6	4
2	1	4	14	13	9
4	3	5	19	16	11
3	2	9	28	26	17

```

-----

```

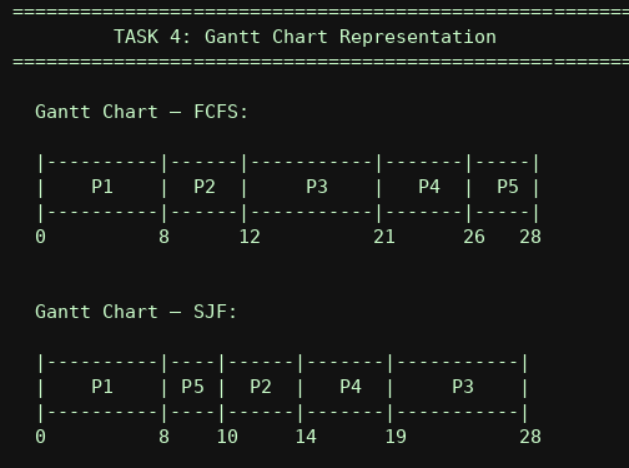
Task 4: Gantt Chart Representation

A text-based Gantt chart shows the execution timeline for both FCFS and SJF, with process labels and time markers.

```

def draw_gantt_chart(gantt, title):
    """Draw a text-based Gantt chart."""
    print(f" Gantt Chart – {title}:")
    top = " |"
    for label, start, end in gantt:
        width = max(len(label), end - start) + 2
        top += "-" * width + "|"
    print(top)
    row = " |"
    for label, start, end in gantt:
        width = max(len(label), end - start) + 2
        row += label.center(width) + "|"
    print(row)
    print(top)
    timeline = " "
    for label, start, end in gantt:
        width = max(len(label), end - start) + 2
        timeline += str(start).ljust(width + 1)
        timeline += str(gantt[-1][2])
    print(timeline)

```



Task 5: Performance Analysis & Comparison

Average Waiting Time and Average Turnaround Time are computed and compared. SJF outperforms FCFS: Avg WT = 8.20 vs 11.40, Avg TAT = 13.80 vs 17.00.

```

def calculate_averages(procs):
    n = len(procs)
    avg_wt = sum(p.wt for p in procs) / n
    avg_tat = sum(p.tat for p in procs) / n
    return avg_wt, avg_tat

def performance_comparison(fcfs_procs, sjf_procs):
    fcfs_avg_wt, fcfs_avg_tat = calculate_averages(fcfs_procs)
    sjf_avg_wt, sjf_avg_tat = calculate_averages(sjf_procs)
    print(f" {'Metric':<30} {'FCFS':>10} {'SJF':>10}")
    print(" " + "-" * 52)
    print(f" {'Average Waiting Time':<30} {fcfs_avg_wt:>10.2f} {sjf_avg_wt:>10.2f}")
    print(f" {'Average Turnaround':<30} {fcfs_avg_tat:>10.2f} {sjf_avg_tat:>10.2f}")
    if sjf_avg_wt < fcfs_avg_wt:
        print(" Better Algorithm: SJF")
    else:
        print(" Better Algorithm: FCFS")

```

```
=====
TASK 5: Performance Analysis & Comparison
=====
```

Metric	FCFS	SJF
Average Waiting Time	11.40	8.20
Average Turnaround Time	17.00	13.80

```
-----
Conclusion:
```

```
Better Algorithm : SJF
Reason           : SJF always picks the shortest available job,
                  minimising the time other processes spend waiting.
                  This greedy choice produces the optimal average waiting
                  time among all non-preemptive algorithms for a given
                  process set. FCFS is simpler but suffers from the
                  convoy effect when a long job arrives early.
```